Setting Up a C Compiler on Windows

WSL2 + Ubuntu: installation and how to run commands (with and without the Ubuntu app)

What you will end up with

A Linux environment running inside Windows, with the GCC compiler, the GDB debugger, Make, and Valgrind. This is the setup used for the forex indicator library and backtester project, because AddressSanitizer and Valgrind (memory bug detectors) work properly on Linux.

Requirements: Windows 10 (version 2004 or later) or Windows 11, an internet connection, and about 2 GB of free disk space.

Part 1: Install WSL2 and Ubuntu (one time only)

Step 1. Right-click the Start button and choose **Terminal (Admin)** or **Windows PowerShell (Admin)**. You can also search for "cmd", right-click Command Prompt, and choose **Run as administrator**.

Step 2. Run:

```
wsl --install
```

This enables the required Windows features and downloads Ubuntu.

Step 3. Restart your PC when it asks.

Step 4. After the restart, the Ubuntu window usually opens by itself. If it does not, open the Start menu and launch **Ubuntu**. Wait a minute while it finishes setting up.

Step 5. Create a Linux username and password when prompted.

> **Note:** When you type the password nothing appears on screen, not even dots. That is normal. Just type it and press Enter.

Administrator rights are needed only for Step 2. From now on you can use a normal (non-admin) window.

Part 2: Install the C tools

Open Ubuntu (see Part 3 or Part 4 for how) and run these two commands:

```
sudo apt update
sudo apt install build-essential gdb valgrind
```

Type your Linux password if asked, and answer **Y** when it asks to continue.

Package	What it gives you
build-essential	gcc (C compiler), make (build tool), and standard C headers
gdb	Debugger: step through code, inspect variables, find crashes
valgrind	Detects memory leaks and invalid memory access

Verify the install:

```
gcc --version
gdb --version
```

```
valgrind --version
```

Part 3: Running commands using the Ubuntu app

This is the simplest method.

Step 1. Open the Start menu, type **Ubuntu**, and click the app. A terminal window opens with a prompt like:

```
yourname@YOURPC:~$
```

Step 2. Create a project folder and move into it:

```
mkdir ~/forex-project
cd ~/forex-project
```

Step 3. Create a test file. The simplest way is the **nano** editor:

```
nano hello.c
```

Paste or type the program below. Save with **Ctrl+O**, press Enter, then exit with **Ctrl+X**.

```
#include <stdio.h>

int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

Step 4. Compile and run:

```
gcc -Wall -Wextra hello.c -o hello
./hello
```

If it prints **Hello, C!**, your setup works.

Step 5. Close the window (or type **exit**) when you are done.

Part 4: Running commands without the Ubuntu app

You can reach the same Ubuntu from other places. Use a **normal** window, no administrator needed.

Method A: Command Prompt or PowerShell

Open Command Prompt or PowerShell, then type:

```
wsl
```

The prompt changes to the Linux one. Now every Linux command works exactly as in Part 3 (**mkdir**, **gcc**, **./hello**, and so on). Type **exit** to return to Windows.

Method B: Run a single command from Windows

You can prefix any command with **wsl** without entering Ubuntu. The command runs inside Linux and the result returns to your Windows window:

```
wsl gcc -Wall -Wextra hello.c -o hello
wsl ./hello
```

Note: Typing **gcc** alone in cmd or PowerShell fails with "gcc is not recognized", because the compiler lives inside Ubuntu, not Windows. Either enter Ubuntu first with **wsl**, or put **wsl** in front of the command.

To start in a specific Linux folder from Windows:

```
wsl --cd ~/forex-project
```

Method C: Windows Terminal

Windows 11 includes Windows Terminal. Open it, click the small down-arrow next to the plus (+) tab, and choose **Ubuntu**. You get a modern terminal with tabs, copy/paste, and Ubuntu one click away.

Method D: VS Code (recommended for daily work)

- Install VS Code from code.visualstudio.com.
- Install the extension called **WSL** (by Microsoft) from the Extensions panel.
- In Ubuntu (via any method above), go to your project folder and run:

```
cd ~/forex-project
code .
```

VS Code opens connected to Linux. Use **Terminal > New Terminal** inside VS Code to get an Ubuntu shell right next to your code. You can edit, compile, and run without leaving the window.

Install the **C/C++** extension too, for syntax highlighting, autocomplete, and debugging.

Part 5: Where to keep your files

Keep your project inside the Linux filesystem, for example **~/forex-project**. Avoid working under **/mnt/c/** (your Windows C: drive from inside Linux). It works, but file access is much slower, which matters when you load millions of rows of price data.

To open your Linux folder in Windows File Explorer, run this from inside the folder in Ubuntu:

```
explorer.exe .
```

Or type this in the File Explorer address bar:

```
\\wsl$\Ubuntu\home\yourname
```

This is handy for copying your downloaded forex CSV files into the project folder.

Part 6: Compiler flags you should always use

```
gcc -Wall -Wextra -g -fsanitize=address,undefined file.c -o program
```

Flag	Purpose
-Wall -Wextra	Turn on warnings that catch many common mistakes
-g	Include debug info so gdb can show your source lines
-fsanitize=address,undefined	AddressSanitizer and UBSan: catch memory errors and undefined behaviour at run time

Check a program for memory leaks with Valgrind (compile with **-g** and without the sanitizer flag first):

```
gcc -Wall -Wextra -g hello.c -o hello
valgrind --leak-check=full ./hello
```

Troubleshooting

Problem	Fix
wsl --install does nothing or shows help text	Your Windows may be outdated. Update Windows, then try again. Make sure you ran it as administrator.
Error about virtualization	Enable virtualization (Intel VT-x or AMD-V / SVM) in your PC's BIOS or UEFI settings, then retry.
Ubuntu does not appear after restart	Open PowerShell and run **wsl --list --verbose**. If Ubuntu is missing, run **wsl --install -d Ubuntu**.
gcc: command not found	You are probably in Windows cmd, not Ubuntu. Type **wsl** first. If you are already in Ubuntu, redo Part 2.
Forgot the Linux password	In PowerShell run **wsl -u root**, then **passwd yourname** to set a new one.
Check that you are using WSL2	Run **wsl --list --verbose** and check the VERSION column shows 2. If it shows 1, run **wsl --set-version Ubuntu 2**.
Slow file access	Move your project from /mnt/c/... into your Linux home folder (~).

Quick reference

I want to...	Command
Install everything (admin, once)	wsl --install
Enter Ubuntu from cmd or PowerShell	wsl
Leave Ubuntu	exit
Run one Linux command from Windows	wsl gcc --version
Compile with warnings	gcc -Wall -Wextra file.c -o name
Run the program	./name
Open the folder in VS Code	code .
Open the folder in File Explorer	explorer.exe .
Shut down all WSL (from PowerShell)	wsl --shutdown

Next step: create **indicators.h** and **indicators.c** with a simple moving average (SMA) function, and a small test program to check it.